

LLMs & VLMs course

Implementing a GPT model from scratch

Juan Irving Vasquez

October 14, 2026

Instituto Politécnico Nacional (IPN)

Contents

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

The transformer block

Coding the GPT model

Generating text

Summary and exercises

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

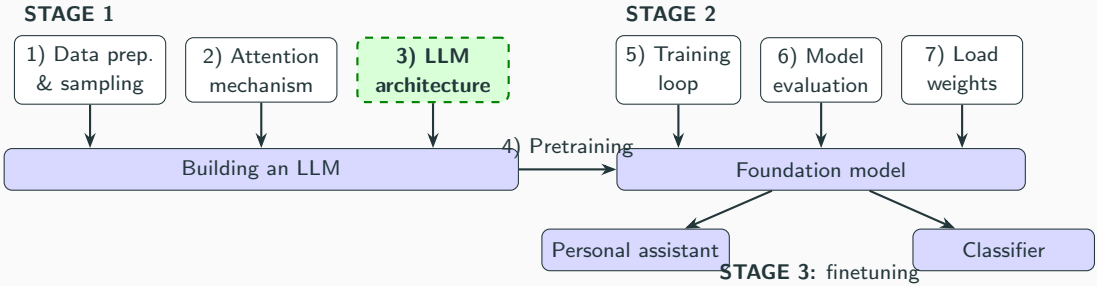
The transformer block

Coding the GPT model

Generating text

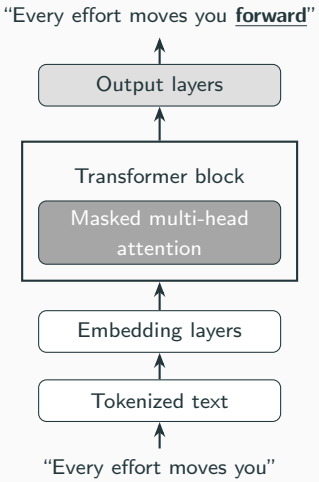
Summary and exercises

Where are we?



Last session: **multi-head attention**. Today: the **other building blocks** and how to assemble them into a GPT-like model.

A mental model of a GPT model



- GPT = **G**enerative **P**retrained **T**ransformer.
- Generates new text **one token at a time**.
- Despite its size, the architecture is simple: most components are **repeated**.
- Today: the **transformer block** and the full model.

Target: GPT-2 small (124M parameters)

- Radford et al., *Language Models are Unsupervised Multitask Learners* (2019).
- Weights are **publicly available** $\Rightarrow$ we can load them later.
- GPT-3 has the same architecture, scaled from 1.5B to **175B** parameters.
- GPT-2 runs on a laptop; GPT-3 would take $\sim$355 years to train on a single V100.

Model	Params	Layers
GPT-2 small	124M	12
GPT-2 medium	355M	24
GPT-2 large	774M	36
GPT-2 XL	1,558M	48
GPT-3	175B	96

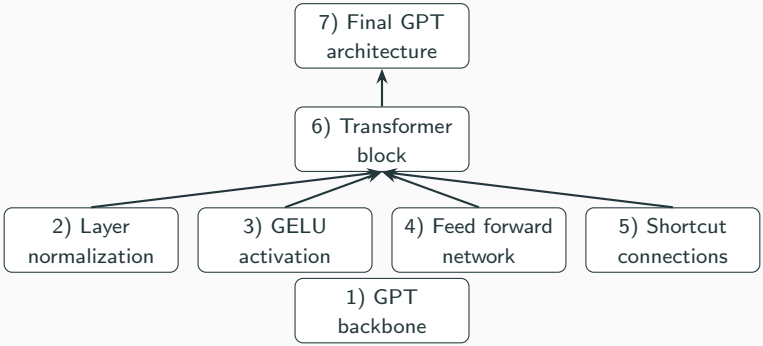
Parameters
Trainable weights. A 2048×2048 layer already has 4,194,304 parameters.

The model configuration

```
GPT_CONFIG_124M = {
        "vocab_size": 50257,     # Vocabulary size
        "context_length": 1024,  # Context length
        "emb_dim": 768,          # Embedding dim
        "n_heads": 12,           # Attention heads
        "n_layers": 12,          # Transformer
             blocks
        "drop_rate": 0.1,        # Dropout rate
        "qkv_bias": False        # Query-Key-Value
              bias
}
```

- `vocab_size`: BPE tokenizer vocabulary (Ch. 2).
- `context_length`: max. input tokens (positional embeddings).
- `emb_dim`: each token $\rightarrow$ 768-d vector.
- `n_heads`: heads in multi-head attention.
- `n_layers`: number of transformer blocks.
- `drop_rate`: 10% of hidden units dropped.
- `qkv_bias`: bias in W^Q, W^K, W^V (off, as in modern LLMs).

Plan for this session



We start with a **placeholder backbone** to see the big picture, then fill in blocks 2–5, assemble block 6, and finish with the full GPT (7).

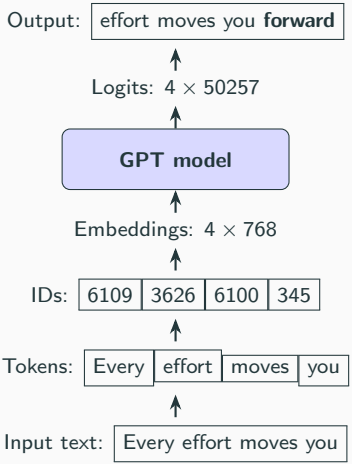
Step 1: A placeholder GPT backbone

```
class DummyGPTModel(nn.Module):
def __init__(self, cfg):
super().__init__()
self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
self.drop_emb = nn.Dropout(cfg["drop_rate"])
self.trf_blocks = nn.Sequential(
*[DummyTransformerBlock(cfg) for _ in range(cfg["n_layers"])])
self.final_norm = DummyLayerNorm(cfg["emb_dim"])
self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

def forward(self, in_idx):
batch_size, seq_len = in_idx.shape
tok_embeds = self.tok_emb(in_idx)
pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
x = self.drop_emb(tok_embeds + pos_embeds)
x = self.trf_blocks(x)
x = self.final_norm(x)
return self.out_head(x)          # logits
```

`DummyTransformerBlock` and `DummyLayerNorm` simply return `x` for now.

Data flow in and out of a GPT model



- Tokenize a batch of 2 texts with `tiktoken` (GPT-2 BPE).
- Input shape: `[2, 4]`
- Output shape: `[2, 4, 50257]`
- One **logit vector per token**, one dimension per vocabulary entry.
- The last row should predict the **next** word.

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

The transformer block

Coding the GPT model

Generating text

Summary and exercises

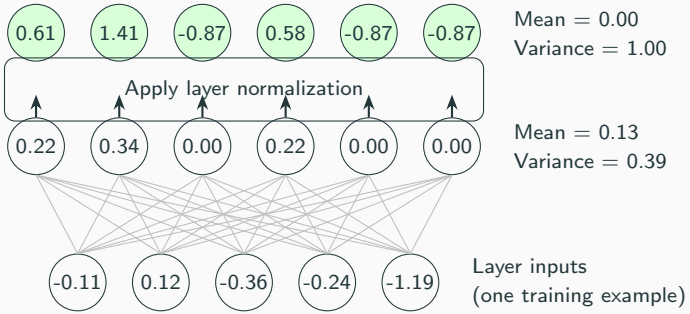
Why normalize?

- Deep networks suffer from **vanishing / exploding gradients** $\Rightarrow$ unstable training.
- **Idea:** rescale the activations of a layer to **mean 0** and **variance 1**.
- Faster convergence and more reliable training.

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \qquad \mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

- Computed over the **feature (embedding) dimension** of each token.
- γ (*scale*) and β (*shift*) are **trainable**.
- ϵ (e.g. 10^{-5}) avoids division by zero.

Layer normalization: an example



Computing mean and variance in PyTorch

```
torch.manual_seed(123)
batch_example = torch.
    randn(2, 5)
layer = nn.Sequential(nn
    .Linear(5, 6), nn.
    ReLU())
out = layer(
    batch_example)



mean = out.mean(dim=-1,
    keepdim=True)
var  = out.var(dim=-1,
    keepdim=True)
out_norm = (out - mean)
    / torch.sqrt(var)
```

`keepdim=True` keeps a `[2,1]` shape instead of `[2]`, so broadcasting works.

- `dim=-1`: last dimension (one statistic **per row**).
- `dim=0`: one statistic **per column**.
- With 3D tensors `[batch, tokens, emb]`, `dim=-1` still normalizes each token's embedding.

	Mean	Variance
Before	0.1324 / 0.2170	0.0231 / 0.0398
After	≈ 0	1.0

A `LayerNorm` module

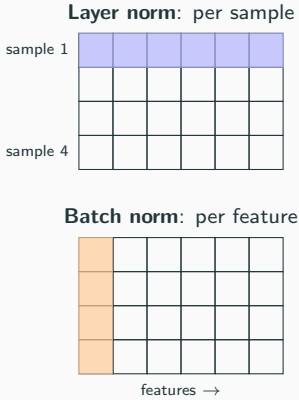
```
class LayerNorm(nn.Module):
def __init__(self, emb_dim):
super().__init__()
self.eps = 1e-5
self.scale = nn.Parameter(torch.ones(emb_dim))    #
    gamma
self.shift = nn.Parameter(torch.zeros(emb_dim))   #
    beta

def forward(self, x):
mean = x.mean(dim=-1, keepdim=True)
var = x.var(dim=-1, keepdim=True, unbiased=False)
norm_x = (x - mean) / torch.sqrt(var + self.eps)
return self.scale * norm_x + self.shift
```

Biased variance (`unbiased=False`)

Divides by n instead of $n-1$ (no Bessel's correction). Negligible for large n, and it matches GPT-2's original TensorFlow implementation $\Rightarrow$ compatible with pretrained weights.

Layer norm vs. batch norm



- **Batch norm** normalizes across the **batch** dimension.
- **Layer norm** normalizes across the **feature** dimension.
- Layer norm is **independent of the batch size**:
  - Works with batch size 1 at inference.
  - Better for distributed training and constrained hardware.

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

The transformer block

Coding the GPT model

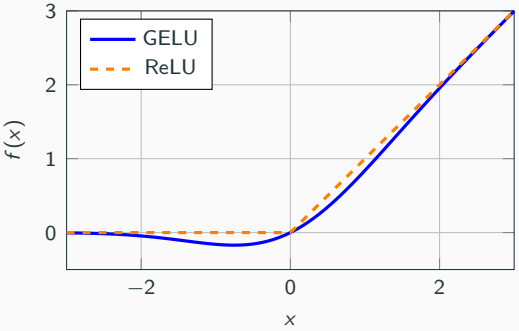
Generating text

Summary and exercises

The GELU activation function

- LLMs use smoother activations than ReLU: **GELU** and **SwiGLU**.
- Exact definition: $\mathrm{GELU}(x) = x\,\Phi(x)$, with Φ the standard Gaussian CDF.
- GPT-2 uses the cheaper approximation:

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1+\tanh\left[\sqrt{\tfrac{2}{\pi}}\left(x+0.044715\,x^3\right)\right]\right)$$



GELU vs. ReLU

ReLU

- Piecewise linear: $\max(0, x)$.
- Sharp corner at $0 \Rightarrow$ harder optimization in very deep networks.
- Zero output (and gradient) for every negative input.

GELU

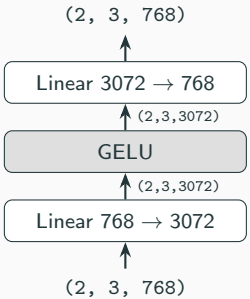
- Smooth and non-linear everywhere.
- Allows more nuanced parameter updates.
- Small **non-zero** output for negative inputs $\Rightarrow$ those neurons still contribute to learning.

The `FeedForward` module

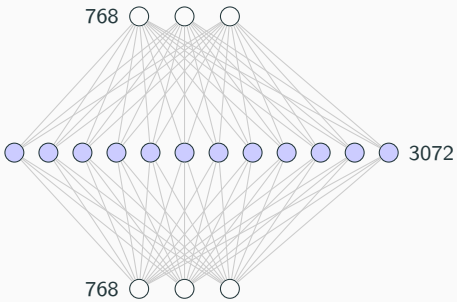
```
class GELU(nn.Module):
def forward(self, x):
return 0.5 * x * (1 + torch.
      tanh(
torch.sqrt(torch.tensor(2.0 /
      torch.pi)) *
(x + 0.044715 * torch.pow(x,
      3))))

class FeedForward(nn.Module):
def __init__(self, cfg):
super().__init__()
self.layers = nn.Sequential(
nn.Linear(cfg["emb_dim"], 4 *
      cfg["emb_dim"]),
GELU(),
nn.Linear(4 * cfg["emb_dim"],
      cfg["emb_dim"]))

def forward(self, x):
return self.layers(x)
```



Expand, activate, contract



- First layer **expands** by a factor of 4 ($768 \to 3072$).
- GELU adds non-linearity in the larger space $\Rightarrow$ **richer representations**.
- Second layer **contracts** back to 768.
- Same input and output shape $\Rightarrow$ blocks can be **stacked** without adjusting dimensions.
- Works for any batch size and number of tokens.

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

The transformer block

Coding the GPT model

Generating text

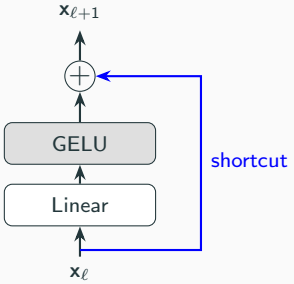
Summary and exercises

Shortcut (skip / residual) connections

- Proposed in **ResNets** (computer vision) to fight **vanishing gradients**.
- Gradients shrink as they propagate backward $\Rightarrow$ early layers barely learn.
- A shortcut **adds the input of a layer to its output**:

$$\mathbf{x}_{\ell+1} = \mathbf{x}_\ell + f_\ell(\mathbf{x}_\ell)$$

- Creates an alternative, shorter path for the gradient: $\frac{\partial \mathbf{x}_{\ell+1}}{\partial \mathbf{x}_\ell} = I + \frac{\partial f_\ell}{\partial \mathbf{x}_\ell}$



Experiment: a 5-layer network with and without shortcuts

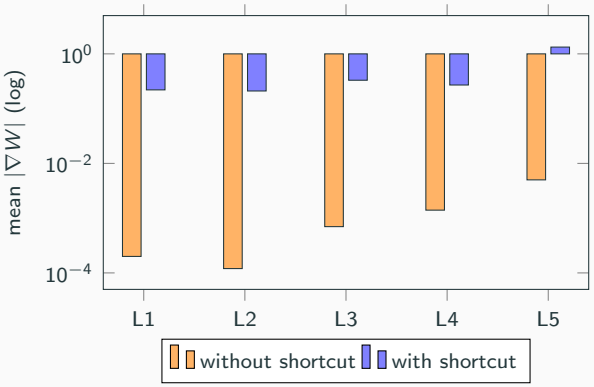
```
class ExampleDeepNeuralNetwork(nn.Module):
def __init__(self, layer_sizes, use_shortcut):
super().__init__()
self.use_shortcut = use_shortcut
self.layers = nn.ModuleList([
nn.Sequential(nn.Linear(layer_sizes[i], layer_sizes[i+1]), GELU())
for i in range(5)])

def forward(self, x):
for layer in self.layers:
layer_output = layer(x)
if self.use_shortcut and x.shape == layer_output.shape:
x = x + layer_output       # shortcut connection
else:
x = layer_output
return x

layer_sizes  = [3, 3, 3, 3, 3, 1]
sample_input = torch.tensor([[1., 0., -1.]])
```

We compute an MSE loss against target 0, call `loss.backward()` and print the **mean absolute gradient** of each layer's weights.

Results: gradients per layer



- **Without** shortcuts: the gradient shrinks from 0.005 (layer 5) to 0.0002 (layer 1) $\Rightarrow$ **vanishing gradient**.
- **With** shortcuts: gradients stay around 0.2–0.3 even in the first layers.
- Shortcuts are a **core building block** of LLMs: they keep gradient flow consistent across dozens of layers.

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

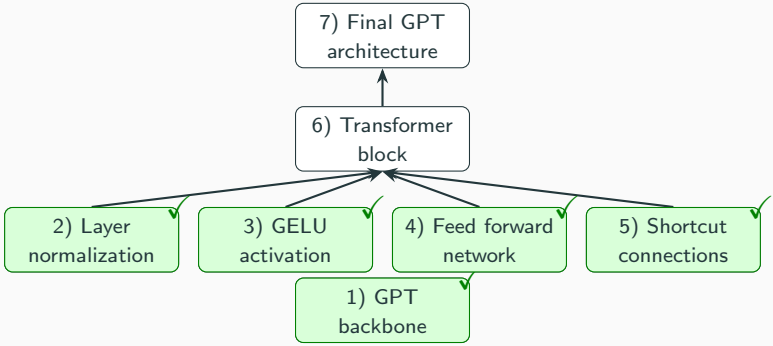
The transformer block

Coding the GPT model

Generating text

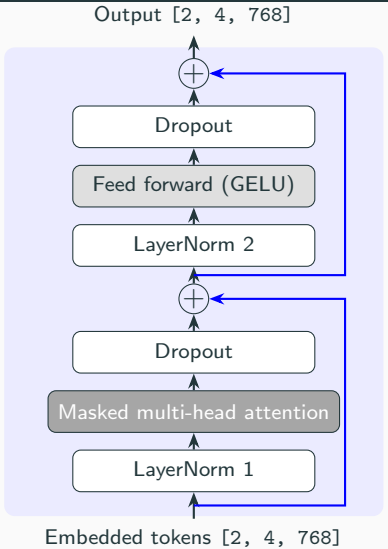
Summary and exercises

Progress so far



Now we combine blocks 2–5 with multi-head attention into a **transformer block**.

Anatomy of the transformer block



- **Self-attention** relates tokens **to each other**.
- **Feed forward** transforms each position **independently**.
- **Pre-LayerNorm**: normalize *before* each sub-layer (GPT-2 and modern LLMs). The original Transformer used *Post-LayerNorm*, with worse training dynamics.
- **Dropout** after each sub-layer for regularization.
- **Shortcut** around each sub-layer.
- Repeated **12 times** in GPT-2 small.

TransformerBlock in code

```
class TransformerBlock(nn
    .Module):
def __init__(self, cfg):
super().__init__()
self.att =
    MultiHeadAttention(
d_in=cfg["emb_dim"],
d_out=cfg["emb_dim"],
block_size=cfg["
    context_length"],
num_heads=cfg["n_heads"],
dropout=cfg["drop_rate"],
qkv_bias=cfg["qkv_bias"])
self.ff = FeedForward(cfg
    )
self.norm1 = LayerNorm(
    cfg["emb_dim"])
self.norm2 = LayerNorm(
    cfg["emb_dim"])
self.drop_resid = nn.
    Dropout(cfg["
    drop_rate"])
```

```
def forward(
    self, x
    ):
# Attention
    sub-
    layer
shortcut = x
x = self.
    norm1(x
    )
x = self.att
    (x)
x = self.
    drop_resid
    (x)
x = x +
    shortcut


# Feed
    forward
     sub-
    layer
shortcut = x
x = self.
    norm2(x
```

Shape preservation

- Input `[2, 4, 768]` $\rightarrow$ output `[2, 4, 768]`.
- This is a **design decision**, not an accident:
  - Blocks can be stacked freely.
  - One output vector per input token (one-to-one).
- The **shape** stays the same, but the **content** changes: each output is a **context vector** that integrates information from the whole (previous) sequence.

$$\underbrace{\mathbf{x}_1, \ldots, \mathbf{x}_n}_{\text{token embeddings}} \xrightarrow{\text{Transformer block}} \underbrace{\mathbf{z}_1, \ldots, \mathbf{z}_n}_{\text{context vectors}}, \qquad \mathbf{x}_i, \mathbf{z}_i \in \mathbb{R}^{768}$$

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

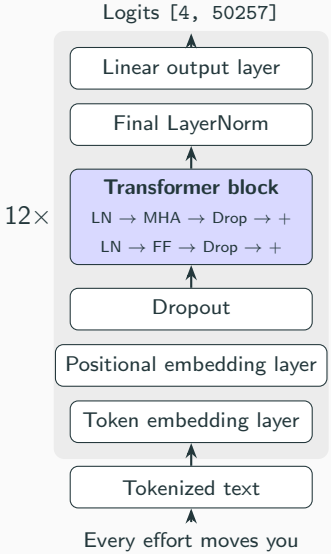
The transformer block

Coding the GPT model

Generating text

Summary and exercises

The full GPT-2 architecture



1. Token IDs → **token embeddings** + **positional embeddings**.
2. Dropout on the embeddings.
3. **12 stacked transformer blocks** (36 in GPT-2 XL*).
4. **Final LayerNorm**.
5. **Linear output head**: 768 → 50,257 (vocabulary size) ⇒ scores for the next token.

*As stated in the book; the original GPT-2 XL has 48 layers (see Exercise 4.2).

GPTModel in code

```
class GPTModel(nn.Module):
def __init__(self, cfg):
super().__init__()
self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
self.drop_emb = nn.Dropout(cfg["drop_rate"])
self.trf_blocks = nn.Sequential(
*[TransformerBlock(cfg) for _ in range(cfg["n_layers"])])
self.final_norm = LayerNorm(cfg["emb_dim"])
self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

def forward(self, in_idx):
batch_size, seq_len = in_idx.shape
tok_embeds = self.tok_emb(in_idx)
pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
x = self.drop_emb(tok_embeds + pos_embeds)
x = self.trf_blocks(x)
x = self.final_norm(x)
logits = self.out_head(x)
return logits
```

Identical to `DummyGPTModel`, with the real `TransformerBlock` and `LayerNorm`.

How many parameters?

```
model =
    GPTModel
    (
    GPT_CONFIG_124M
    )
total_params
     = sum(
    p.numel
    ()
for p in
    model.
    parameters
    ())
# Total
    number
    of
    parameters
    :
    163,009,536
```

Weight tying

The original GPT-2 **reuses the token embedding weights** in the output layer.

$$163{,}009{,}536 - 38{,}597{,}376 = \mathbf{124{,}412{,}160}$$

- Tying reduces memory and compute.
- Separate layers often train better; most modern LLMs do so.
- We revisit tying when loading OpenAI's weights.

Memory footprint

- Each parameter is a 32-bit float = **4 bytes**.

$$\frac{163{,}009{,}536 \times 4 \text{ bytes}}{1024^2} \approx \mathbf{621.83\ MB}$$

- Even a "small" LLM needs significant storage.
- By the same rule, a 175B-parameter model needs $\approx$ **700 GB** in FP32 (just for the weights).
- This motivates lower precision formats (FP16, BF16, INT8, ...).

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

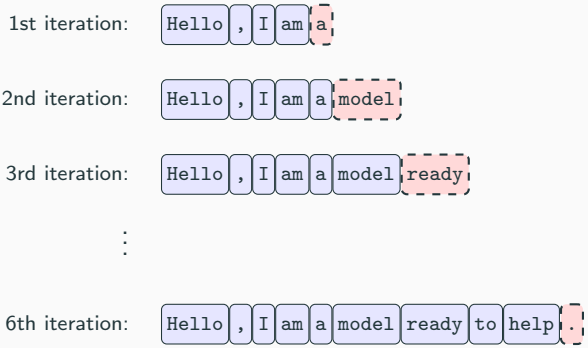
The transformer block

Coding the GPT model

Generating text

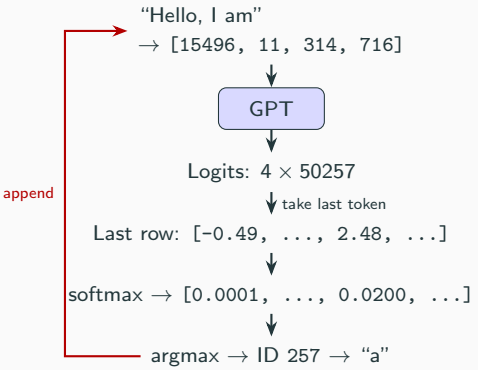
Summary and exercises

Generating one token at a time



The predicted token is **appended** to the input; the context **grows** at each iteration (autoregressive generation).

One generation step in detail



1. Encode text into token IDs.
2. Run the model → logits.
3. Keep the **last** vector (next-token prediction).
4. Softmax → probabilities.
5. Argmax → token ID.
6. Append and repeat.

generate_text_simple

```
def generate_text_simple(model, idx, max_new_tokens, context_size):
# idx: [batch, n_tokens] array of token IDs in the current context
for _ in range(max_new_tokens):
idx_cond = idx[:, -context_size:]          # crop to supported context
with torch.no_grad():
logits = model(idx_cond)               # [batch, n_tokens, vocab]
logits = logits[:, -1, :]                  # only the last time step
probas = torch.softmax(logits, dim=-1)     # [batch, vocab]
idx_next = torch.argmax(probas, dim=-1, keepdim=True)  # [batch, 1]
idx = torch.cat((idx, idx_next), dim=1)    # append
return idx
```

- Softmax is **monotonic**: argmax over logits gives the same result. Kept for intuition.
- Always choosing the most likely token = **greedy decoding**. Sampling strategies (temperature, top-k) come in the next chapter.

Trying it out

```
start_context = "Hello, I am"
encoded = tokenizer.encode(start_context)          # [15496, 11, 314, 716]
encoded_tensor = torch.tensor(encoded).unsqueeze(0) # add batch dim -> [1, 4]

model.eval()   # disable dropout
out = generate_text_simple(model=model, idx=encoded_tensor,
max_new_tokens=6,
context_size=GPT_CONFIG_124M["context_length"])
print(tokenizer.decode(out.squeeze(0).tolist()))
```

Output

```
Hello, I am Featureiman Byeswickattribute argue logger Normandy ...
```

Gibberish! The model has **random weights**: it has not been **trained** yet $\Rightarrow$ next chapter: pretraining.

Roadmap

From attention to a full GPT

Layer normalization

Feed forward network with GELU

Shortcut connections

The transformer block

Coding the GPT model

Generating text

Summary and exercises

Summary

- **Layer normalization** stabilizes training by giving each layer's outputs a consistent mean and variance.
- **GELU** is a smooth activation used in the **feed forward** module (expand $\times 4$, contract).
- **Shortcut connections** skip layers and mitigate vanishing gradients.
- **Transformer blocks** combine masked multi-head attention and feed forward networks with Pre-LayerNorm, dropout and shortcuts.
- **GPT models** stack many identical transformer blocks; the same `GPTModel` class covers 124M to 1.5B parameters.
- **Text generation** decodes the output one token at a time; without training the output is incoherent.

Practice: Implementing a GPT model